From Architecture to Terraform

Letting AI Do the Boring Parts





MOMENTUM
Developer Conference
powered by The Circuit

The Problem With Infrastructure Today

Manual infrastructure assembly is slow, repetitive, and full of governance overhead — the bottleneck is human, not technical.

Slow Builds

Weeks-long delivery cycles stall product teams before a single resource is deployed.

Repetition

The same Terraform patterns written from scratch, again and again, across every project.

Governance Overhead

Every deployment requires review, approval, and audit trails, adding friction at every step.

Human Bottlenecks

Engineers spend time on assembly work, not architecture; the highest-value work waits.



Value Stream Mapping Session

The team saw the waste clearly — and one person decided to do something about it.

- **End-to-end visibility** — the team mapped their service delivery process from first request to final deployment
- **Waste made visible** — every handoff, every wait, every repetitive step exposed on the board
- **Frustration was high** — the team knew they could move faster; the data confirmed it
- **"There has to be a better way"** — a shared desire for acceleration, waiting for someone to act

The Bold Claim

"I can take our two-week infrastructure build process and shrink it to two days."

The room went silent.

The Bold Claim

Reaction in the Room

Skepticism and curiosity filled the room in equal measure, but nobody dismissed it outright.

Skepticism

"Two weeks to two days? That's not a process improvement — that's magic."

Curiosity

"How would that even work?"

The Key Question

"Is that even possible?"

The Silence

Nobody laughed. That itself was telling.



Why the Claim Seemed Plausible

The conditions for automation were already in place; I saw a pattern everyone else had missed.

Cloud Patterns Repeat

Resource groups, APIM, Service Bus, and Container Apps appear in nearly every architecture

Terraform Is Repetitive

Most of the code is assembly of known, approved modules written from scratch each time

Docs Encode Decisions

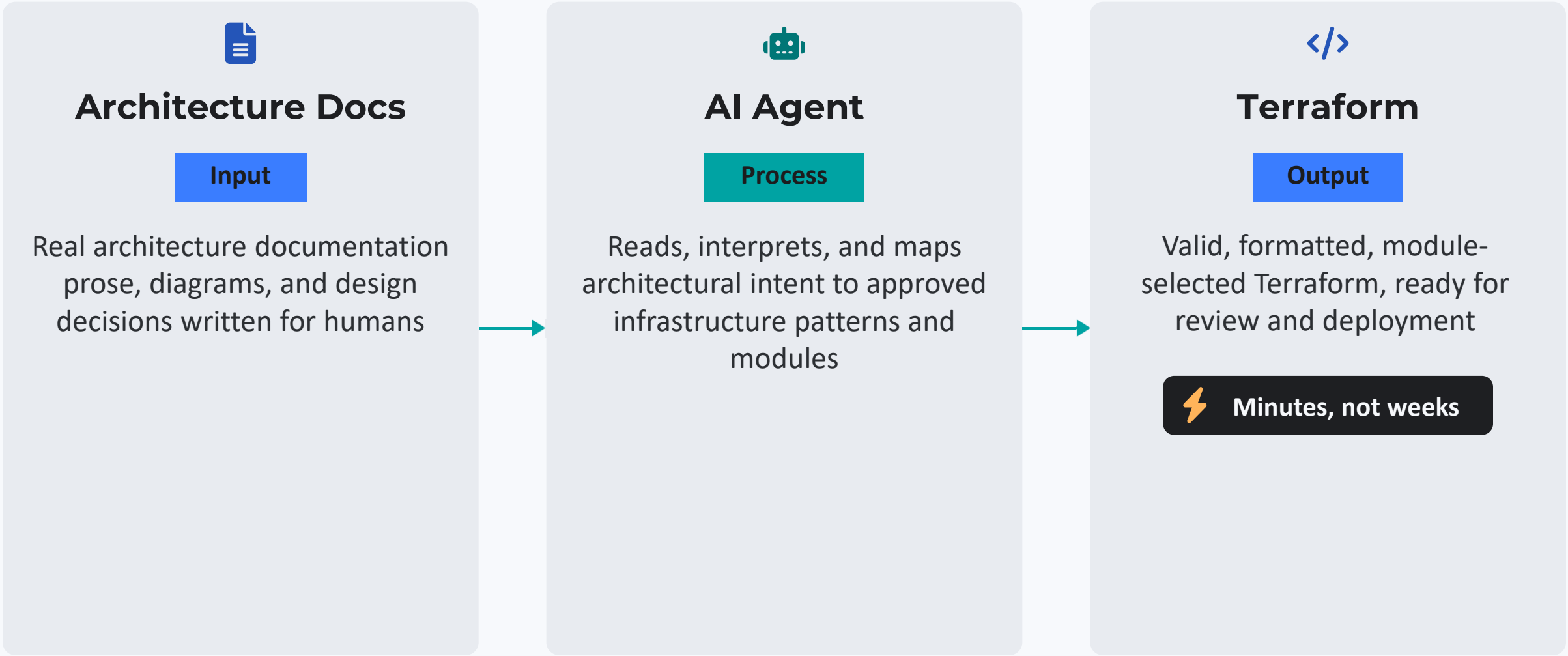
Architecture documentation already captures intent. The decisions are written down and waiting

AI Excels at Patterns

AI reads, interprets, and maps structure better than humans at scale, perfect for this task

The First Prototype

Alex built a simple pipeline: feed architecture docs in, get Terraform out.



AI Architecture Recognition

What AI Recognized

The AI didn't just read the doc — it understood the architecture's anatomy. [2]



Resource Groups

Logical boundaries and naming scope



APIM

API Management gateway patterns



Service Bus

Async messaging topology



Cosmos DB

NoSQL data tier with partition strategy



Container Apps

Containerized compute layer

Opinionated Modules

The AI didn't generate raw Terraform — it selected from a governed library of opinionated modules.

Raw Terraform

Guardrails

None — any misconfiguration can slip through undetected

Standards

Manual — naming, tagging, and regions vary per author

Security

Opt-in — private endpoints, RBAC, encryption not guaranteed

Source of truth

Blank slate — every author starts from scratch

Opinionated Module

Guardrails built in

No misconfigured resources slipping through

Standards enforced

Naming conventions, tagging, region constraints

Security defaults applied

Private endpoints, RBAC, encryption at rest

Team decision encoded

Each module captures a decision — not a blank slate

Output

Governed

Every resource traces back to an approved module

Consistent

Standards applied uniformly across every deployment

Secure by default

No opt-in required — security is baked in

Auditable

Clear lineage from architecture intent to deployed resource

Terraform Output

The output wasn't just syntactically correct — it was production-ready and standards-aligned. [2]

main.tf – generated by AI pipeline

```hcl
# Auto-generated – standards-aligned Terraform
module "resource_group" {
  source = "./modules/rg-opinionated"
  name   = "rg-contoso-prod-eus"
  tags   = local.standard_tags
}

module "apim" {
  source = "./modules/apim-opinionated"
  name   = "apim-contoso-prod"
  sku    = "Premium_1"
  private_ep = true
}

module "service_bus" {
  source = "./modules/sb-opinionated"
  name   = "sb-contoso-prod-eus"
  sku    = "Premium"
}
```

Valid

`terraform validate`

✓ passed on first run

Deployable

`terraform plan`

Clean, expected diff — no surprises

Standards-Aligned

Naming, tagging, and module selection matched team conventions exactly

Fast

Generated in a fraction of the time a human would take



The Prototype Works

Team Reaction

The team went from skeptical to intrigued; the prototype had earned its shot.

Surprise
"This actually looks like our Terraform."

Excitement
"If this holds up, it changes everything."

Cautious Optimism
"Let's run it again on a different architecture."

The Verdict
"This might actually work."



The Hero's Early Victory

For one bright moment, the bold claim looked not just possible but inevitable.

- **The prototype worked**
- **The team believed**
- **Two weeks → two days** felt within reach
- This is the moment in every hero's journey when the quest seems won; just before the real challenge appears

Running the Same Architecture Again

The same input. A different output. The crack in the foundation.

RUN 1 Terraform Output — Baseline

```
module "api_gateway" {
source  = "./modules/apim"
 name = "apim-prod-westeu"
 sku = "Premium"
 regions = [ "westeurope" ]
}
module "service_bus" {
source  = "./modules/servicebus"
 name = "sb-prod-westeu"
 sku = "Premium"
}
module "container_app" {
source    = "./modules/containerapps"
 name = "ca-prod-westeu"
 min_replicas = 2
 max_replicas = 10
}
# Tags applied to all resources
locals {
 common_tags = {
 env = "prod"
 team = "platform"
}
}
```

VS

RUN 2 Terraform Output — Diverged

```
# Tags declared first — structure shifted
locals {
 common_tags = {
 environment = "prod"
 team = "platform"
}
}
module "api_gateway" {
source  = "./modules/apim"
name    = "apim-westeu-prod"
 sku = "Premium"
 regions = [ "westeurope" ]
}
module "container_app" {
source    = "./modules/containerapps"
name      = "containerapps-prod-we"
 min_replicas = 2
 max_replicas = 10
}
module "service_bus" {
source  = "./modules/servicebus"
name    = "sbus-westeurope-prod"
 sku = "Premium"
}
```

Same doc
byte-for-byte identical

Same agent
same model, same prompt

Different Terraform
naming, ordering, structure

Still correct
but not reproducible

What Changed?

Nothing in the input changed — but the AI made different choices every time.

Run 1 — Original Output

```
module "apim" {
 source = "./modules/apim"
 name = "apim-prod-we-001"
 sku = "Premium_1"
}

module "servicebus" {
 source = "./modules/servicebus"
 namespace = "sb-prod-we-001"
 sku = "Premium"
}

# Container Apps environment
module "container_apps" {
 source = "./modules/containerapps"
 env_name = "cae-prod-we"
}
```

Run 2 — Different Output

```
# API Management gateway
module "apim" {
 source = "./modules/apim"
 name = "apim-prod-westeurope"
 sku = "Premium"
}

module "container_apps" {
 source = "./modules/containerapps"
 env_name = "cae-prod-we-001"
}

module "servicebus" {
 source = "./modules/servicebus"
 namespace = "sb-prod-westeurope-ns"
 sku = "Standard"
}
```

● Unchanged ● Changed by AI

Naming | Ordering | Formatting | Comments | Structure

Why This Is a Problem

Non-determinism isn't just inconvenient — in infrastructure, it's a governance crisis. [3]

CI/CD Diffs Explode

Pipelines flag changes that aren't real changes — noise overwhelms signal and blocks deployment confidence.

Reproducibility Breaks

You can't rebuild yesterday's infrastructure exactly — every run is a one-way door with no reliable return path.

Governance Becomes Unpredictable

Auditors can't compare runs meaningfully — compliance evidence loses integrity when outputs vary without cause.

Standards Drift

Subtle variations accumulate over time, eroding consistency — the codebase quietly diverges from what was agreed.

AI Strengths vs Weaknesses

Alex had discovered AI's fundamental asymmetry — brilliant at one job, unreliable at the other.

Strength

Where AI Excels

- **Reading Context**
  Absorbs prose, diagrams, and implicit architectural intent
- **Recognizing Patterns**
  Identifies resource types, topology, and relationships across diverse documentation
- **Mapping Intent**
  Translates fuzzy human language into structured, actionable output
- **Understanding Nuance**
  Handles ambiguity and convention without explicit instruction

Weakness

Where AI Fails

- **Producing Identical Output**
  Cannot guarantee the same result across runs, even with identical input
- **Byte-for-Byte Consistency**
  Naming, ordering, and formatting vary between generations
- **Strict Reproducibility**
  CI/CD pipelines and governance audits require exact repeatability — AI cannot deliver this

Determinism Is Non-Negotiable

Infrastructure isn't a creative medium — it demands the same answer every single time. [3]

Stability

Infrastructure requires stability; every drift is a potential incident

Predictable Diffs

Engineers must trust that `git diff` shows real intent changes — not AI variation

Repeatable Builds

The same YAML must produce the same Terraform today, tomorrow, and next quarter

The Turning Point

"

AI is great at interpretation . Not at deterministic generation .

This wasn't a failure — it was a **discovery**

The question shifted: **"Are we using AI for the right job?"**

Reframing the Problem

The solution wasn't a better AI — it was a better division of labor.

AI was doing two jobs simultaneously

Job 1

Interpretation

Reading architecture docs, understanding intent, recognizing patterns

AI is excellent here

AI did BOTH

Job 2

Generation

Producing Terraform, enforcing structure, guaranteeing consistency

AI is unreliable here

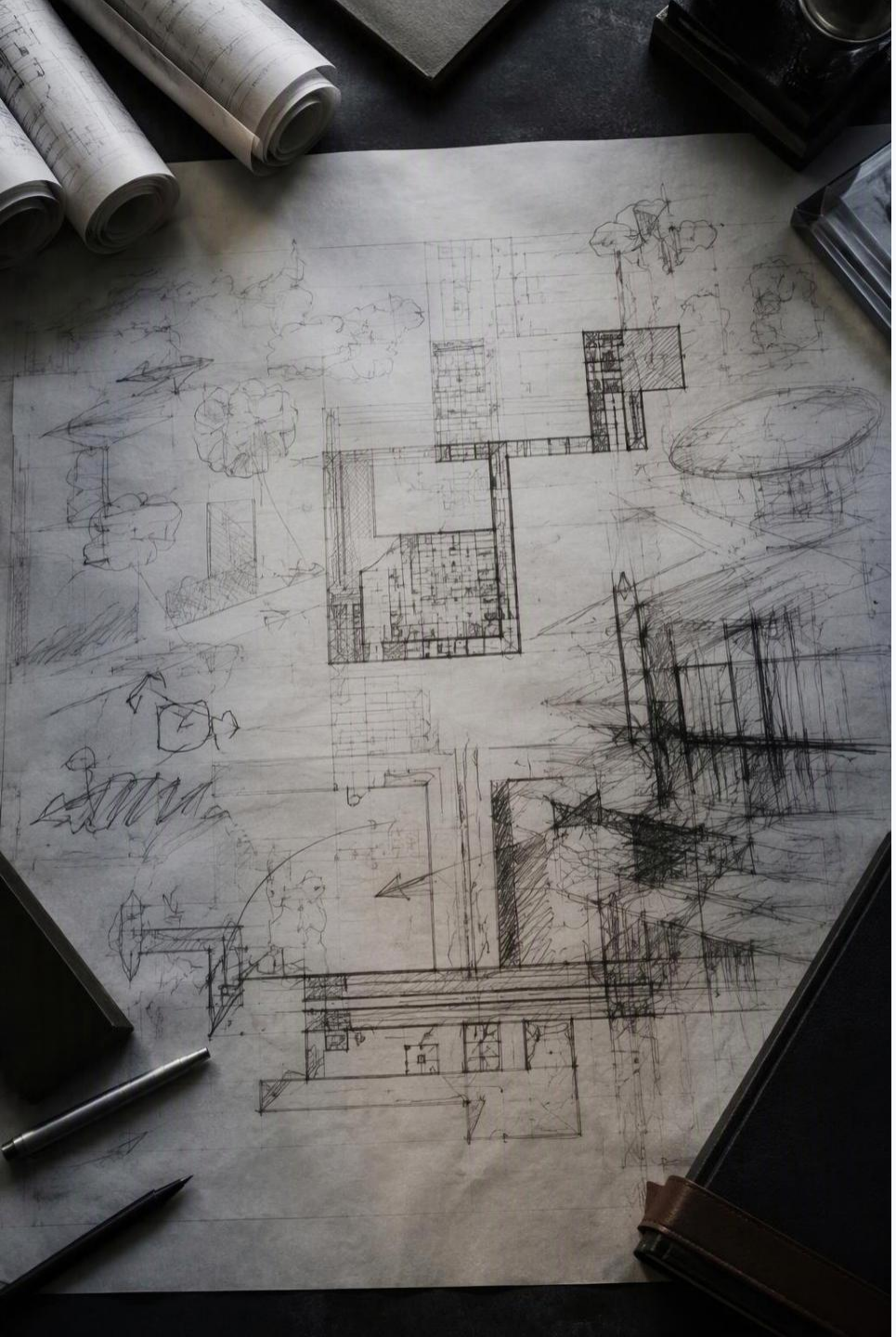
Only one of these jobs actually needs AI

What if AI shouldn't generate Terraform at all? [2]

A pivot in thinking — not a defeat

AI handles **interpretation** something else handles **generation**

What does that **unlock** ?



Architecture Intent Is Fuzzy

Human architecture documentation is rich, ambiguous, and full of implied decisions — exactly what AI is built for.

- **Human language:**"We need an API gateway that's secure and scalable" — no specifics, high intent
- **Diagrams:**Boxes and arrows that imply topology without specifying parameters
- **Assumptions:**Decades of team conventions embedded in how things are named and organized
- Architecture intent is inherently interpretive — and that is fine

Terraform Must Be Deterministic

Terraform is not a creative artifact; it is a contract with your infrastructure.

Architecture Doc

Fuzzy by Design

Human Language

"Secure and scalable API gateway" — high intent, no specifics

Implied Topology

Boxes and arrows that suggest structure without specifying parameters

Interpretive Intent

Decisions embedded in team conventions — requires reading between the lines

Variability Is Fine Here

AI thrives on ambiguity — this is exactly what it is built for

VS

Terraform Output

Deterministic

Byte-for-Byte Identical

The same intent must always produce exactly the same code — no exceptions

Governed

Every resource choice, parameter, and tag must be traceable to a decision

Predictable

CI/CD pipelines, audit logs, and deployment histories depend on stability

Variability Is a Crisis

Any drift in Terraform output is a governance failure — not acceptable

The New Model

The breakthrough was architectural: split the pipeline into two distinct, purpose-built stages.



Stage 1 — AI

- **Reads** architecture documentation, interprets intent
- **Produces YAML** — variable, intelligent, contextual

Stage 2 — PowerShell

- **Reads YAML** , generates Terraform deterministically
- **Fixed output** — deterministic, governed, consistent

YAML as a Machine-Readable Contract

YAML became the bridge between human intent and deterministic code — structured, versioned, and governed. [2]

infrastructure-contract.yaml — Deterministic Contract

```yaml
# Machine-readable infrastructure contract
schema_version : "2.1"
environment : "production"

naming :
prefix : "contoso-prod"
region : "eastus2"

modules :
- type : "apim"
sku : "Premium_1"
private_endpoint : true
- type : "container_apps"
replicas : 3
cpu : "0.5"

tags :
cost_center : "platform-eng"
classification : "confidential"
team : "infra-automation"
```

Contract Properties

Structured

Every field explicitly defined — no ambiguity, no interpretation required

Deterministic

The same YAML always produces the same Terraform — guaranteed

Governed

Schema is version-controlled, reviewed, and enforced across the team

Machine-Readable

PowerShell reads it without interpretation — pure data, pure output

What YAML Encodes

YAML captures every decision that previously lived in a developer's head — or got lost in translation. [1]

Naming

Resource name patterns, prefixes, suffixes, environment tags

Networking

VNet, subnet, peering, private endpoints

Modules

Which opinionated module to invoke for each component

Parameters

Size, SKU, replica count, retention, throughput

Tags

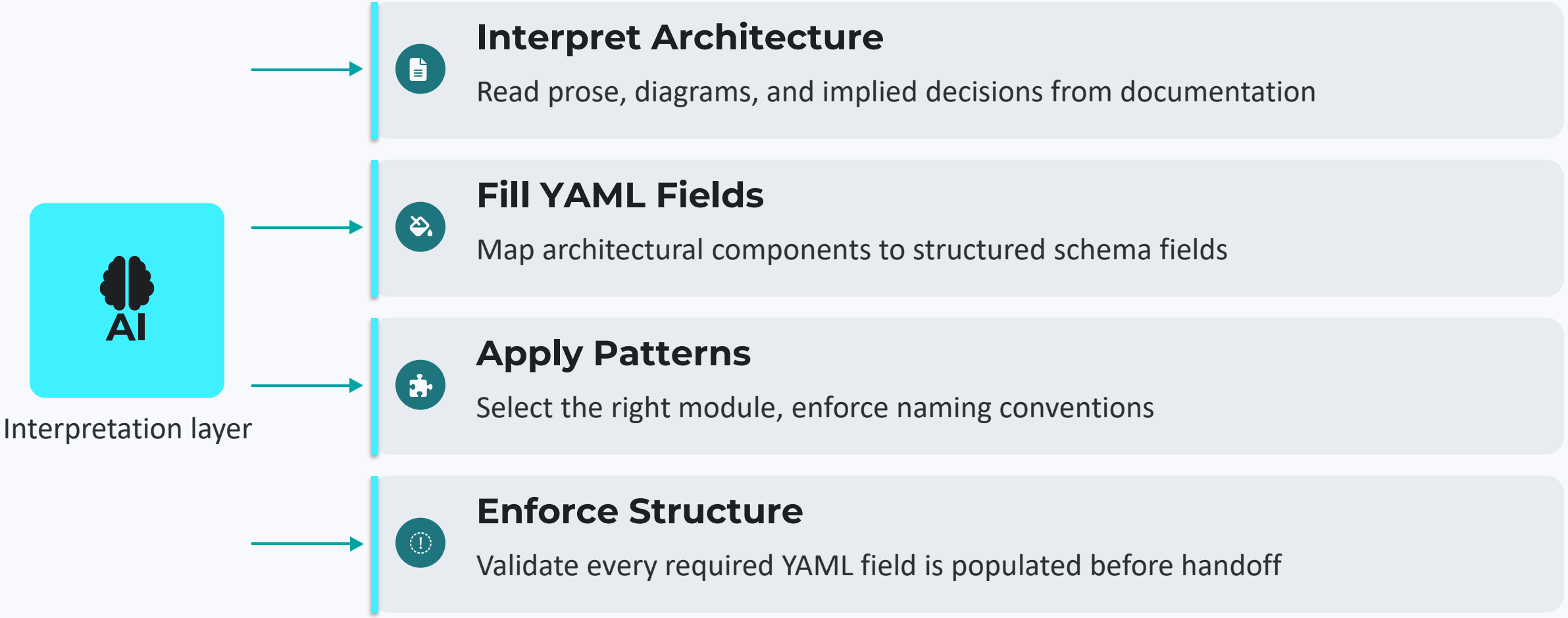
Cost center, team, environment, classification

Conventions

Region, redundancy zone, backup policy

AI's New Job

AI's scope narrowed — and it immediately became more reliable.



Stage 2 — Deterministic Generation

PowerShell's Job

PowerShell doesn't interpret — it executes. The same input, the same output, every time.

New-TerraformFromYaml.ps1

```powershell
# Read YAML - parse the machine-readable contract
$config = ConvertFrom-Yaml ( Get-Content $YamlPath -Raw)

# Generate Terraform deterministically - field by field
foreach ( $resource in $config .resources) {
 $tfBlock = New-TerraformBlock `
-Module $resource .module `
-Name $resource .name `
-Params $resource .parameters `
-Tags $config .global.tags
 $output += $tfBlock
}

# Write output - identical every run
$output | Set-Content $OutputPath -Encoding UTF8

# Orchestrate: module selection, naming, file output
Invoke-TerraformCoordinator -Config $config -Out $OutputPath
```

Read YAML

Parse the machine-readable contract field by field — no interpretation required

Generate Terraform

Every module call, every parameter, every block — scripted and fixed

PowerShell Coordinator

Orchestrates module selection, naming injection, and file output

Identical Output Every Run

Speed Improvement

The bold claim became reality — not through AI writing Terraform, but through AI removing the bottleneck.

Terraform Generation

10s

From YAML to deployable Terraform

End-to-End Cycle

2 weeks → 2 days

Infrastructure delivery cycle time

What changed

- Engineers no longer spend time on assembly; they spend time on architecture decisions
- AI removed the bottleneck; interpretation is fast, deterministic generation handles the rest
- The boring parts are gone; what remains is higher-value work

The Breakthrough Moment

The right tool for each job, and suddenly the whole system worked.

AI

FUZZY WORK

Handles what's ambiguous

- Interpretation — reading context and nuance
- Pattern recognition across diverse docs
- Intent mapping — architecture to structured output
- Variability is acceptable here

Where AI excels

Scripts

DETERMINISTIC

Handles what must be exact

- YAML-to-Terraform generation
- Same input → same output, every run
- Governed modules enforced by design
- Consistency is non-negotiable here

Byte-for-byte reliable

Engineers

HIGHEST VALUE

Handles what requires judgment

- Architecture decisions — trade-offs and constraints
- Governance review — oversight and approval
- Exception handling — edge cases and escalations
- Focus on impact, not assembly

Freed from the boring parts

Live Demo

Demo Overview

Here is Alex's final workflow — live, from architecture document to deployable Terraform. [2]

1

Architecture Doc

Human Intent

Real documentation, the kind a team actually writes — prose, diagrams, decisions

`input`

AI Agent

2

AI → YAML

Deterministic Contract

Structured output from AI interpretation that is versionable, reviewable, and governable

`structured`

PowerShell

3

PowerShell → Terraform

Identical Output

Governed modules, correct naming, CI/CD-ready, identical every run

`deployable`

Demo — Step 1 of 3

Step 1: Architecture Doc

This is the input — human-written, intent-rich, and deliberately imprecise.

infra-architecture.md

```
// Architecture Document – Platform Services v2

## Overview
We need a secure, scalable API gateway that exposes our
microservices to external consumers with rate limiting and auth .

## Messaging
Use async messaging to decouple order processing from
fulfillment. The message broker must guarantee delivery.

## Data Tier
Store event records in a globally distributed NoSQL database .
Partitioned by tenant ID . Low latency reads required.

## Compute
Deploy services as containers – they need to scale to zero
and autoscale under peak load. Keep costs low.
```

Real Document

Not a template. Written by architects for humans — not for machines.

Contains

Service descriptions, component relationships, networking intent, security requirements.

Written For Humans

High intent, low specificity — exactly what AI must interpret before YAML is produced.

AI Input

AI reads this next — extracting structure from intent.

Step 2 Demo Walkthrough

AI → YAML

AI reads the architecture and produces a structured, deterministic contract — no freeform Terraform.

infrastructure.yaml — Deterministic Contract

```yaml
# AI-generated from architecture doc
environment: "production"
region: "eastus2"
naming_prefix: "contoso-prod"

components:
  api_management:
    module: "apim-standard-v2"
    sku: "StandardV2"
    private_endpoint: true
  service_bus:
    module: "servicebus-premium"
    tier: "Premium"
    messaging_units: 1
  container_apps:
    module: "containerapps-env"
    replicas: 3
tags:
  cost_center: "platform-eng"
  classification: "internal"
```

Interprets Intent

AI maps architecture prose to structured YAML schema fields — naming, modules, parameters, tags

Explicitly Defined

Every field declared — no ambiguity, no missing values, no freeform generation

Version-Controlled

This YAML is reviewable, auditable, and committed to source control before Terraform runs

Repeatable Output

Run AI again on the same doc — same YAML. The contract doesn't drift between runs

Step 3 of 3

PowerShell → Terraform

Identical Output

PowerShell reads YAML and generates Terraform — byte-for-byte identical, every single run. [2]

main.tf — generated by PowerShell coordinator

```
# Auto-generated — identical every run

module "apim" {
source = "./modules/apim"
name = "apim-prod-eastus-001"
resource_group = var.resource_group_name
sku_name = "Developer_1"
tags = local.common_tags
}

module "servicebus" {
source = "./modules/servicebus"
name = "sb-prod-eastus-001"
sku = "Standard"
private_endpoint_enabled = true
}

module "container_apps" {
source = "./modules/container_apps"
name = "ca-prod-eastus-001"
replicas = 3
}
```

terraform validate — PASS | terraform plan — clean diff

Pure Execution

PowerShell reads YAML field by field — no interpretation, pure execution

Governed Modules

Terraform output uses governed modules, correct naming, enforced standards

Run 1 = Run 2 = CI/CD

Run it once — same Terraform. Run it again — same Terraform. Every time.

CI/CD Stability

Diffs reflect real intent changes — not AI variation noise

When AI Works

Use AI where variability is acceptable and interpretation is the challenge.



Interpretation

Reading human-language architecture docs and extracting structured meaning



Pattern Recognition

Identifying resource types, topology, and relationships across diverse documentation styles



Reducing Repetitive Work

Eliminating the manual translation from architect intent to module selection



Encoding Intent

Capturing implicit team conventions and applying them consistently to YAML fields

When AI Fails

Avoid AI where determinism is required, the cost of variability is too high.



Do Not Use AI For

Deterministic Output

AI cannot guarantee byte-for-byte identical artifacts across runs. Each execution may produce subtly different results — even with identical inputs and prompts.

Avoid



Do Not Use AI For

Strict Governance

Where every parameter must be traceable and auditable, variability is a compliance risk. Governance frameworks require provable, repeatable decisions — not probabilistic ones.

Avoid



Do Not Use AI For

Reproducible Artifacts

CI/CD pipelines, infrastructure audits, and disaster recovery all depend on exact repeatability. AI-generated artifacts introduce drift that breaks these contracts.

Avoid

AI didn't replace Terraform authors. It replaced the boring parts.



Architecture
AI interprets intent

YAML
Deterministic contract

Terraform
Identical output, every run

- AI accelerates interpretation of reading docs, mapping intent, and filling structured contracts
- Deterministic scripts guarantee consistency. PowerShell generates the same Terraform every run
- Engineers focus on higher-value work, architecture decisions, governance, and judgment calls
- Platform teams reduce repetition and increase governance at scale

Find me online

Thank you

Chad Green

Senior Systems Architect

Email

chadgreen@chadgreen.com

Web Site

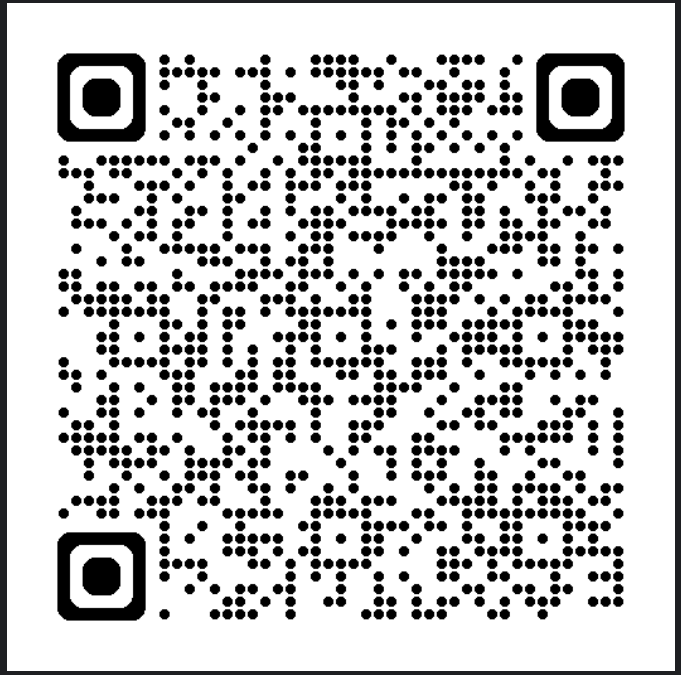
ChadGreen.com

Twitter

@ChadGreen

LinkedIn

ChadwickGreen



Slides & Demo Code

Scan to get the repo